```python
"""
Data models for the Weather Dashboard application.

"""

from datetime import datetime, date
from dataclasses import dataclass, field
from typing import Optional


# WeatherRecord Immutable weather record at a specific time

@dataclass
class WeatherRecord:
    """Represents a single weather observation or forecast entry."""

    city: str
    country: str
    recorded_date: date
    temperature_c: float
    feels_like_c: float
    humidity_pct: int
    wind_speed_kmh: float
    weather_description: str
    is_forecast: bool = False



    @property
    def temperature_f(self) -> float:
        """Convert Celsius to Fahrenheit."""
        return round(self.temperature_c * 9 / 5 + 32, 1)

    @property
    def summary_tuple(self) -> tuple:
        """
        Return a lightweight tuple summary.

```

```
        Format: (city, date_str, temp_c, description)
        """
        return (
            self.city,
            self.recorded_date.strftime("%Y-%m-%d"),
            self.temperature_c,
            self.weather_description,
        )

    def __str__(self) -> str:
        kind = "Forecast" if self.is_forecast else "Current"
        return (
            f"[{kind}] {self.city}, {self.country} | "
            f"{self.recorded_date} | "
            f"{self.temperature_c}°C (feels {self.feels_like_c}°C) | "
            f"Humidity: {self.humidity_pct}% | "
            f"Wind: {self.wind_speed_kmh} km/h | "
            f"{self.weather_description}"
        )


# FavoriteCity - persisted city entry


@dataclass
class FavoriteCity:
    """Represents a city saved by the user."""

    name: str
    country: str
    latitude: float
    longitude: float
    added_on: date = field(default_factory=date.today)


    @property
    def coordinates(self) -> tuple:
```

```python
        """Return (latitude, longitude) tuple."""
        return (self.latitude, self.longitude)

    def to_dict(self) -> dict:
        """Saves plain dict for JSON storage."""
        return {
            "name": self.name,
            "country": self.country,
            "latitude": self.latitude,
            "longitude": self.longitude,
            "added_on": self.added_on.isoformat(),
        }

    @classmethod
    def from_dict(cls, data: dict) -> "FavoriteCity":
        """Saves from dict (loaded from JSON)."""
        return cls(
            name=data["name"],
            country=data["country"],
            latitude=data["latitude"],
            longitude=data["longitude"],
            added_on=date.fromisoformat(data["added_on"]),
        )

    def __str__(self) -> str:
        lat, lon = self.coordinates
        return f"{self.name}, {self.country} (lat={lat}, lon={lon}) — saved {self.added_on}"
```